

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Permissions, Ownership, and ACLs

Unit 4 · Linux (10-151-105)



Learning Outcomes

By the end of this unit, you will be able to:



Configure standard permissions, ownership, and attributes

Read and set rwx access for the user, group, and others, reassign ownership, and apply file attributes such as the immutable flag.



Configure special permissions and troubleshoot access

Apply SUID, SGID, and the sticky bit where standard permissions fall short, and work through common permission failures methodically.



Configure Access Control Lists (ACLs)

Grant fine-grained access to additional users and groups beyond the single owner and single group that standard permissions allow.

Why Permissions Matter

Every action on a Linux system is checked against the identity of the user who requested it. Permissions are how the system decides who may read a file, change it, run it, or enter a directory. Set them too loosely and you expose data; set them too tightly and people can't do their jobs.

This unit takes the account structure you built in Unit 3 and makes it operational: the users and groups now become the basis for real access control.



The Principle of Least Privilege

Grant each user the minimum access needed to do the job — and nothing more.

- Kai needs to read a report → grant read only, not read-write-execute.
- Joseph creates his own files → he works from a standard account, not root.

Every extra permission is a door you'll later have to watch. This principle guides every decision in this unit.

LESSON 3.1

Configure Standard Linux Permissions



Three Identities: User, Group, Others

Standard permissions apply access to exactly three identities. Each resource (file or directory) has one owner and one associated group.



User (u)

The owner — by default, the account that created the resource. Reassigned with chown.



Group (g)

A single group associated with the resource. Lets many users share one access level.



Others (o)

Everyone else — not the owner and not in the group. Usually the most restricted.



Rocky and Ubuntu agree here

Permissions, ownership, and ACLs behave the same in both ecosystems — the model below is identical on Rocky Linux 10 and Ubuntu Server 24.04.

Three Access Levels — and They Differ for Directories

Read (r), write (w), and execute (x) mean something different depending on whether they apply to a file or a directory. This is where many learners stumble.

Permission	On a file	On a directory
read (r)	View the file's contents	List the directory's contents (the names inside)
write (w)	Change or save the file	Create, rename, and delete entries — but only with x as well
execute (x)	Run it as a program or script	Enter the directory with cd and access items inside

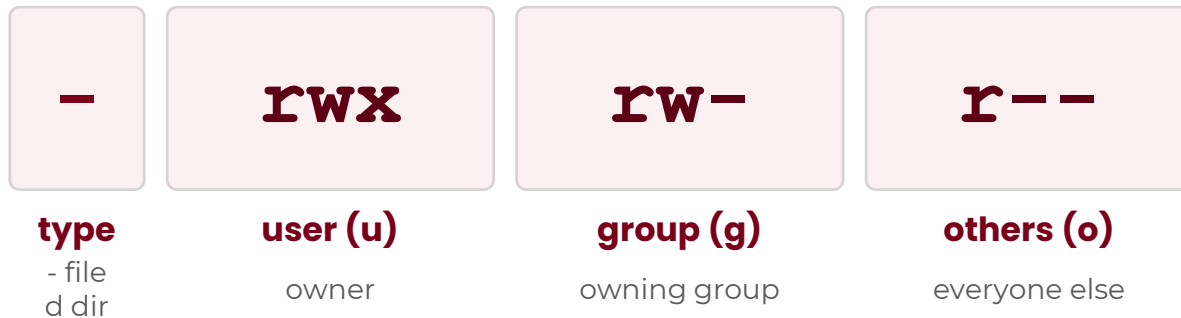


The classic gotcha

On a directory, x is the right to enter it. Without execute on a directory, you cannot cd into it — even if you can read the names. And write on a directory is what lets you delete files in it, regardless of who owns those files.

Reading ls -l: The 10-Character String

ls -l ("list in long format") shows the permission string. Ten characters break into four meaningful parts:



Within each rwx group the order is always read, write, execute. A dash (-) is a placeholder for a permission that is not granted.

```
$ ls -l
-rwxrw-r-- 1 root  sales   2048  Jun 23 09:14  fileA
  ^ user=rwx  ^ group=rw-  ^ others=r--  (owner root, group sales)
```

Permission-String Examples

Practice translating between the symbolic string and what it actually allows:

String	Meaning
<code>-rwxrw-r--</code>	File. Owner: read/write/execute. Group: read/write. Others: read.
<code>-r--r-----</code>	File. Owner: read. Group: read. Others: no access.
<code>drwxr-xr-x</code>	Directory. Owner: full. Group and others: enter and list, but not modify.
<code>dr-xr-x---</code>	Directory. Owner & group: enter and list. Others: no access at all.



Remember the leading character

The very first character is not a permission — it tells you the resource type. A dash means a file; d means a directory. Everything after it is the three rwx triplets.

Changing Permissions: chmod — Absolute Mode

Absolute mode uses octal (base-8) numbers. Each permission has a value; add them up for each identity, then write the three digits as owner-group-others.

Permission	Value
read (r)	4
write (w)	2
execute (x)	1

Worked examples

```
7 = 4+2+1 → rwx
6 = 4+2   → rw-
5 = 4+1   → r-x
```

```
$ chmod 755 dirA → rwxr-xr-x
$ chmod 640 file1 → rw-r-----
```



Only the owner or root

Just like changing permissions in general, only the file's owner or the root user (via sudo) may run chmod on a resource. Use the -R option to apply changes recursively through a directory tree — carefully.

chmod — Symbolic Mode

Symbolic mode reads more like a sentence: who, what change, which permissions.

Context

u g o a

user, group, others, all

Operator

+ - =

add, remove, set exactly

Attribute

r w x

read, write, execute

```
$ chmod u+rw,g+rw myfile      add read+write for owner and group
$ chmod o-r report.txt        remove read from others
$ chmod u=rwx,g=rx,o=rx dirA  set exact permissions (same as 755)
```

Watch the spacing: one space after chmod, one space before the filename — no spaces anywhere else.

Two Modes, Same Result

Neither mode is "better" — but you must be able to read and use both, because documentation and other admins use either one.

Intended access	Absolute	Symbolic
Owner rwx · group r-x · others r-x	chmod 755	chmod u=rwx,g=rx,o=rx
Owner rwx · group none · others none	chmod 700	chmod u=rwx,g=,o=
Owner rw · group r · others none	chmod 640	chmod u=rw,g=r,o=
Owner rw · group none · others none	chmod 600	chmod u=rw,g=,o=

Tip: pick the mode that feels natural to you for everyday work, but practice reading the other.

Ownership: chown and chgrp

Ownership decides who controls a resource's permissions. By default the creator owns it; only root can hand ownership to someone else. Changing ownership requires sudo.

chown — change owner and/or group

```
chown newowner file
chown newowner:newgroup file
chown :newgroup file
chown -R newowner dir/

$ sudo chown student5 project1.txt
```

chgrp — change group only

```
chgrp groupname file

$ sudo chgrp sales report.txt

(an alternative to
chown :sales report.txt)
```



The colon tells chown what you mean

owner:group changes both. :group (colon first) changes only the group. owner alone changes only the owner. Forgetting the colon is the most common chown mistake.

Default Permissions: umask

Rather than set permissions by hand on every new file, umask defines the defaults. The twist that confuses everyone: umask specifies what to remove from the starting defaults, not what to grant.

	Owner	Group	Others
Default for new files (666)	rw-	rw-	rw-
umask 022 removes...	0	2 (write)	2 (write)
Result	rw-	r--	r--



Why files never come out executable

Because umask subtracts from 666 (files) and 777 (directories), it can clear bits but never add them. New files don't get the execute bit from umask — you grant that explicitly with chmod when a file needs to run. Set umask per session, or persist it in ~/.bashrc or ~/.profile.

File Attributes: Beyond Permissions

Attributes are a separate layer from rwx permissions. They control what the system itself is allowed to do with a file — independent of who owns it. Examples include append-only, automatic compression, recoverable-on-delete, and immutability.

lsattr — list attributes

```
$ lsattr file2
----i-----e---- file2

i = immutable    e = extents
```

chattr — change attributes

```
$ sudo chattr +i file3
$ sudo chattr -i file3

+ adds    - removes    (root needed)
```



Attributes ≠ permissions

A file can have wide-open rwx permissions and still refuse to change because an attribute blocks it. That disconnect is exactly why attributes catch people off guard — and it leads straight into our next topic.

Deep Dive: The Immutable Flag

The immutable flag is an attribute that locks a file or directory completely. While it is set, the file cannot be modified, renamed, or deleted — and that restriction applies to everyone, including the root user.

It exists for files that are sensitive, important, and not expected to change: a careless command or a runaway process simply cannot touch them. Root must deliberately remove the flag first.



Even root is blocked

`rm`, `mv`, and `write` all fail on an immutable file until `chattr -i` removes the flag.



Spotting it

In `lsattr` output, a lowercase `i` in the attribute list means the immutable flag is set. A directory can hold a mix of mutable and immutable files.

The Immutable Flag in Practice

Set it, confirm it, then prove it's locked

```
$ sudo chattr +i /etc/resolv.conf      # set immutable (root required)
$ lsattr /etc/resolv.conf             # confirm
----i-----e---- /etc/resolv.conf
$ sudo rm /etc/resolv.conf             # blocked!
rm: cannot remove 'resolv.conf': Operation not permitted
$ sudo chattr -i /etc/resolv.conf      # remove the flag to allow changes again
```



Troubleshooting tip

"This file can't be deleted and it should be deletable." Check for the immutable flag with `lsattr`, then clear it with `chattr -i`.



Protecting a file

"This file keeps getting deleted by mistake." If it shouldn't change, set `chattr +i` to lock it down.

Troubleshooting Standard Access

Most access problems are one of two things: a misapplied permission, or a misunderstood identity. Work the problem in order.

- Confirm the identity. Is the user actually in the group they think they're in? Check with `id` or `groups`; fix with `usermod -aG group user`.
- Inspect the permissions. Run `ls -l`. Are owner/group/others set the way the task needs? Don't forget the parent directories — you need `x` on every directory in the path.
- Test as the user. Use `su` to step into their shell and reproduce the failure directly.
- Reapply carefully. Adjust with `chmod`, double-checking absolute vs symbolic syntax, then re-test.



Same on Rocky and Ubuntu

`chmod`, `chown`, `chgrp`, `umask`, `lsattr`, and `chattr` work identically across Rocky Linux 10 and Ubuntu Server 24.04. The one earlier-unit difference to keep in mind: the administrative group is `wheel` on Rocky and `sudo` on Ubuntu.

Lesson 3.1 Review

- Three identities (user, group, others) × three access levels (read, write, execute).
- x means different things for files (run) and directories (enter).
- Read permission strings from `ls -l`; the first character is the type.
- Set permissions with `chmod` in absolute (octal) or symbolic mode.
- Reassign ownership with `chown` / `chgrp` (root + sudo).
- Control defaults with `umask` (it masks, it doesn't grant).
- Lock critical files with the immutable flag (`chattr +i`, even root is blocked).

LESSON 3.2

Configure Special Linux Permissions



Deep Dive: Why Special Permissions Exist

Sometimes standard rwx isn't enough. Consider passwd: an ordinary user must change their own password, which means writing to /etc/shadow — a file only root can touch. Special permissions solve this by letting a program run with the file owner's or group's privileges, briefly and in a controlled way.

There are three special bits — and they ride on a fourth octal digit

Special bit	Octal	Symbolic	What it does
SUID	4	u+s	Run an executable as its owner
SGID	2	g+s	Run as the group; on a dir, new files inherit the group
Sticky	1	+t	Only the owner (or root) may delete items in a directory



The leading digit

Standard permissions are three octal digits (e.g., 755). A fourth, leading digit carries the special bits — so chmod 4755 is SUID (4) plus rwxr-xr-x (755).

SUID — Set User ID

SUID makes an executable run with the privileges of its owner, not the user who launched it. The textbook example is `passwd`: it's owned by root, so any user who runs it temporarily gains the access needed to update the password files — and nothing more.

How it looks and how to set it

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x root root passwd
    ^ s in the owner's
    execute position

$ sudo chmod u+s prog    (or 4755)
```



Read the s

When SUID is set, the owner's execute slot shows `s` instead of `x`. A capital `S` means SUID is set but execute is not — usually a mistake worth a second look.



Use it sparingly

An SUID-root program is a privilege gateway. If it has a flaw, an attacker may use it to act as root. Apply SUID only to trusted executables, and audit for unexpected SUID files as a hardening habit.

SGID — Set Group ID

SGID has two uses. On an executable, it runs with the file's group privileges. Far more useful day-to-day: on a directory, it forces every new file and subdirectory to inherit the directory's group — perfect for shared team folders.

On a shared directory

```
$ sudo chmod g+s /sales-tools
(or chmod 2775 /sales-tools)

$ ls -ld /sales-tools
drwxrwsr-x sales /sales-tools
    ^ s in the group's
    execute position
```



Why teams love it

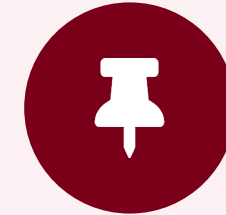
Normally a new file gets the creator's primary group. With SGID on the folder, everything created there belongs to the shared group automatically — so colleagues keep access without anyone changing groups by hand.

Note: it affects new items only, not files already in the folder or moved in.

Deep Dive: The Sticky Bit

Picture a shared directory where everyone needs to create files — like `/tmp`. To create files, users need write on the directory. But write on a directory also lets them delete anything in it, including other people's files. That's the problem the sticky bit fixes.

With the sticky bit set, a user can only delete or rename files they own (and root can always manage them). Everyone keeps the rest of their write access.



Your files are yours

In a sticky directory, only an item's owner — or root — can delete or rename it, even though everyone can still create files.



It's a directory protection

The sticky bit protects the contents of a directory. Modern kernels ignore it on individual files — to protect specific files, set it on the directory that holds them.

Setting and Reading the Sticky Bit

Octal value 1; symbolic +t

```
$ sudo chmod +t /shared
(or chmod 1777 /shared)

$ ls -ld /shared
drwxrwxrwt /shared
    ^ t in the others'
    execute position
```



/tmp is the live example

Run `ls -ld /tmp` on any Linux box and you'll see the trailing `t`. That single bit is what keeps a shared scratch directory from becoming a free-for-all.

You see	It means
t	Sticky set, and others have execute
	Clearing it Remove the sticky bit the same way as the other special bits: <code>chmod -t</code> (symbolic) or set the leading octal digit to 0.

Reading Special Bits in ls -l

Special bits don't get their own columns — they reuse the execute positions. This overload is what makes them tricky to read.

Bit	Appears in...	Lowercase (x also set)	Uppercase (x not set)
SUID	owner's execute slot	s (e.g. -rwsr-xr-x)	S
SGID	group's execute slot	s (e.g. -rwxr-sr-x)	S
Sticky	others' execute slot	t (e.g. drwxrwxrwt)	T



The one rule that unlocks it

Find which execute position the letter sits in: owner = SUID, group = SGID, others = sticky. Lowercase means execute is also on; uppercase means the special bit is set but execute is off.

Setting Special Bits with Absolute Mode

In absolute mode, the special bits live in a leading fourth digit. You must still write all three standard digits — even when you're only changing the special bit.

Command	Leading digit	Effect
<code>chmod 4755 prog</code>	4 = SUID	SUID + <code>rwxr-xr-x</code>
<code>chmod 2775 teamdir</code>	2 = SGID	SGID + <code>rw-rwxr-x</code> (group inheritance)
<code>chmod 1777 /shared</code>	1 = sticky	Sticky + <code>rw-rwxrwx</code> (protected shared dir)
<code>chmod 0755 prog</code>	0 = none	Clears special bits, leaves <code>rwxr-xr-x</code>



Don't clobber the standard bits

Run `ls -l` before and after. It's easy to change the standard permissions by accident when you're focused on the special digit — especially in absolute mode.

Troubleshooting Special Permissions

Same method as standard permissions, with the special bits added to your checklist. A very common ticket: files created in a shared folder get the creator's group instead of the team's group.

The shared-folder fix

```
# group not inheriting?  
$ sudo chmod g+s /sales-tools  
# then re-test by creating a file
```

- Confirm identities and group memberships first.
- Verify SUID is on the right executables.
- Verify SGID is on directories that need group inheritance.
- Verify the sticky bit where shared deletes must be controlled.
- Check `ls -l` before and after every change.



Mind the syntax

SUID is `u+s`, SGID is `g+s`, sticky is `+t`. Mixing these up is the most frequent special-permission slip.

Lesson 3.2 Review

- Special permissions step in when standard rwx can't express the need.
- SUID (4) runs an executable as its owner — the passwd model.
- SGID (2) runs as the group, and on a directory makes new files inherit the group.
- The sticky bit (1) lets only an item's owner (or root) delete it in a shared directory.
- Special bits ride on a leading fourth octal digit (e.g. 4755).
- Read them in the execute positions: owner = SUID, group = SGID, others = sticky.

LESSON 3.3

Configure Access Control Lists



Why ACLs? The Limit of Standard Permissions

Standard permissions can describe exactly one owner, one group, and everyone else. But real situations are messier: what if User1 needs read-write while User2 needs read-only on the same file? Or GroupA needs full access while GroupB needs read-execute? Standard permissions can't say that. ACLs can.



ACLs extend, not replace

An ACL adds extra permission entries on top of the standard owner/group/others. The standard rules still apply — ACLs layer additional users and groups with their own access levels onto the same resource.

Typical use cases

- sales needs rwx, marketing needs r-x — same directory.
- sales-managers can edit a file; sales can only read it.
- A code reviewer gets read-only on another developer's file.

Viewing ACLs: getfacl

getfacl shows the full picture — owner, group, standard permissions, any special-bit flags, and every individual user or group entry.

```
$ getfacl project2.txt
# owner: atanaka
# group: sales
user::rw-
user:student9:r--      <- extra ACL entry
group::r--
mask::r--
other::---
```



Look for the +

ls -l does not list ACL entries — but it flags their presence. If a resource has an ACL, its permission string ends with a + instead of a plain character.

```
$ ls -l project2.txt
-rw-r--r--+ 1 atanaka sales project2.txt <- the trailing + means an ACL is set
```

Configuring ACLs: setfacl

setfacl writes ACL entries. The format names a user or a group, then the permissions.

```
u:username:permissions
g:groupname:permissions

setfacl -m u:kgarcia:rwX fileA
```

Option	Action
-m	Modify / add entries
-x	Remove a specific entry
-b	Remove all ACL entries
-R	Apply recursively



Two formats, one pattern

Start with u: for a user or g: for a group, add the name, then the rwx string. Combine entries with commas to set several at once.

```
$ setfacl -m u:atanaka:rwX,g:sales:rw fileA      # a user and a group at once
$ setfacl -x u:jdeng fileA                       # remove an entry
```


ACL Worked Examples

These are the scenarios standard permissions simply cannot express:

Goal	Command
Give user kgarcia full access to fileA	setfacl -m u:kgarcia:rwX fileA
Give group engineering full access to fileA	setfacl -m g:engineering:rwX fileA
sales-managers edit, sales read-only	setfacl -m g:sales-managers:rw,g:sales:r file
Remove user jdeng's entry from fileA	setfacl -x u:jdeng fileA



Standard rules still apply

ACL entries are checked alongside the standard owner/group/others permissions, not instead of them. If access seems wrong, review both with `ls -l` and `getfacl`.

Default ACLs and a Dual-Distro Note

Default ACLs on directories

A directory can carry default ACL entries that are automatically applied to anything created inside it later — a convenient way to make a whole tree share a consistent access policy without revisiting each new file.



Rocky and Ubuntu: identical commands, one setup difference

getfacl and setfacl behave the same on both systems. The practical difference is availability: a minimal Rocky Linux 10 install may need the acl package added (`sudo dnf install acl`), while Ubuntu Server 24.04 typically includes it. Filesystems must be mounted with ACL support, which is the default on modern ext4 and XFS.



Security angle

ACLs are powerful, but every extra entry is another grant to track. Use them when the standard model genuinely can't express the need — and audit them with getfacl the same way you'd audit standard permissions.

Troubleshooting ACLs

- Display the current entries with `getfacl` and confirm they match what's intended.
- Confirm the user signed in with an identity that should have access; adjust memberships with `usermod` if needed.
- Check for conflicts between ACL entries and standard permissions — review both with `ls -l` and `getfacl`.
- Watch for surprises from recursive (`-R`) changes that cascaded deeper into a tree than expected.
- Fix incorrect entries with `setfacl -m`, or clear them with `-x / -b`.



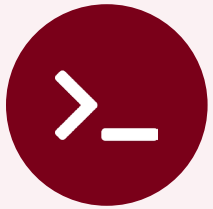
Two sources of truth

When access looks wrong on an ACL-enabled resource, you have two places to look — the standard permissions and the ACL. Check both before changing anything.

Lesson 3.3 Review

- Standard permissions allow only one owner, one group, and others — ACLs go beyond that.
- ACLs add per-user and per-group entries on top of standard permissions; they don't replace them.
- `getfacl` displays entries; a trailing `+` in `ls -l` signals an ACL is present.
- `setfacl -m` adds or changes entries; `-x` and `-b` remove them.
- Directories can carry default ACLs that new contents inherit.
- On a minimal Rocky install, add the `acl` package; Ubuntu Server usually ships it.

Hands-On Lab



NETLAB+ Lab: Permissions, Ownership, and ACLs

In your pod you'll set standard permissions and ownership, apply the immutable flag, configure SUID/SGID and the sticky bit on shared directories, and grant fine-grained access with ACLs — on both Rocky Linux 10 and Ubuntu Server 24.04.

- Start from the WK04_PERMISSIONS snapshot — every attempt begins from the same known-good state.
- Your work is graded on the end state of each system, not a written description.
- Capture the verification output before your reservation ends; changes do not persist.
- See the lab document for full instructions, the rubric, and submission steps.

Security Focus



SECURITY FOCUS

Permissions Are the Front Line of Local Access Control

- Lead with least privilege: grant the minimum access a task requires, then stop.
- A single wrong bit — a world-writable config, an unexpected SUID-root binary — is a real exposure. Audit for them.
- Use the immutable flag to protect critical files from accident and tampering alike.
- Apply the sticky bit to every shared, writable directory so users can't delete each other's work.
- Treat ACL entries like any grant: necessary, documented, and periodically reviewed.

Why it matters: Permissions are the most-used security control on a Linux host. Get them right and most local attacks have nowhere to go; get them wrong and nothing else you harden will hold.

Key Takeaways

- `rw` for user, group, and others is the foundation — and `x` means "execute" on a directory.
- `chmod` sets permissions in absolute or symbolic mode; `chown` / `chgrp` set ownership.
- `umask` masks the defaults for new files; it can clear bits but never grant them.
- The immutable flag (`chattr +i`) locks a file against everyone, including root.
- SUID, SGID, and the sticky bit extend the model; read them in the execute positions.
- ACLs add per-user and per-group access on top of standard permissions, viewed and set with `getfacl` / `setfacl`.
- Everything in this unit works identically on Rocky Linux 10 and Ubuntu Server 24.04.

Questions?

Unit 4 • Permissions, Ownership, and ACLs